

Reviewer Pack — Symmetric Power Partition Dominance in Permanent vs Determin...

Entry 567149797f34 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 04:01:42 UTC

Symmetric Power Partition Dominance in Permanent vs Determinant

Entry ID: 567149797f34 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 04:01:42 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): 3-SAT Instances

Statement:

For random 3-SAT instances with n variables, the number of times the partition (n) appears in the symmetric power decomposition of the permanent polynomial exceeds that in the determinant polynomial by $\Omega(\sqrt{n})$ for all $n \leq 40$.

Rationale (proposer’s reasoning):

Schur-Weyl duality decomposes tensor powers into irreps; permanent’s symmetric nature creates more dominant partitions than determinant’s alternating structure, revealing algebraic asymmetry central to GCT’s permanent vs determinant separation.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 01b7f3c89ef7fa41

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    def random_3sat_instance(n: int, m: int) -> list:
        variables = set(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, 3)
            clauses.append(clause)
        return clauses

    def matrix_from_3sat(instance: list, n: int) -> list:
        matrix = [[0] * n for _ in range(n)]
        for clause in instance:
            for var in clause:
                if var > 0:
                    matrix[var - 1][var - 1] += 1
                else:
                    matrix[-var - 1][-var - 1] -= 1
        return matrix

    def determinant(matrix: list) -> int:
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            sign = (-1) ** (j % 2)
            det += sign * matrix[0][j] * determinant(submatrix)
        return det

    def permanent(matrix: list) -> int:
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        perm = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            sign = (-1) ** (j % 2)
            perm += sign * matrix[0][j] * permanent(submatrix)
        return abs(perm)
```

```

def count_partition_occurrences(matrix, partition):
    n = len(matrix)
    if n != len(partition):
        return 0
    count = 0
    for perm in set(permutations(range(n))):
        product = 1
        for i, p in enumerate(perm):
            product *= matrix[i][p]
        if sorted([product] * partition[i]) == sorted(matrix):
            count += 1
    return count

def permutations(lst):
    if len(lst) == 0:
        return []
    if len(lst) == 1:
        return [lst]
    l = []
    for i in range(len(lst)):
        m = lst[i]
        remLst = lst[:i] + lst[i+1:]
        for p in permutations(remLst):
            l.append([m] + p)
    return l

n = random.randint(5, 40)
m = random.randint(3 * n, 6 * n)
instance = random_3sat_instance(n, m)
matrix = matrix_from_3sat(instance, n)

det_value = determinant(matrix)
perm_value = permanent(matrix)

det_partition_count = count_partition_occurrences(matrix, [n])
perm_partition_count = count_partition_occurrences(matrix, [n])

metric_name = "partition_count_difference"
metric_value = perm_partition_count - det_partition_count
instances_tested = 1
conjecture_holds = metric_value >= math.sqrt(n)
counterexample = "" if conjecture_holds else f"Partition count difference {metric_value} < sqrt(n)"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30)) + list(map(random.randint, range(2, 30), range(2, 30)))
    results = []
    for seed in seeds:

```

```

    random.seed(seed)
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction=1.000")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Partition count difference < sqrt(n)\" first_failing_seed={first_failing_seed}")

def primes(n):
    sieve = [True] * (n+1)
    for x in range(2, int(n**0.5) + 1):
        if sieve[x]:
            for i in range(x*x, n+1, x):
                sieve[i] = False
    return [x for x in range(2, n+1) if sieve[x]]

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ecd9910.py", line 115, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ecd9910.py", line 86, in run_trial
    instance = random_3sat_instance(n, m)
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ecd9910.py", line 22, in random_3sat_instance
    clause = random.sample(variables, 3)
                ~~~~~^
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to invalid population type in random sampling, preventing data collection. | next: Fix the population sequence issue in random_3sat_instance by converting variables to a list

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	81630
2	preregistration	ollama_remote	qwen3:8b	0	0	24518
3	novelty	ollama_remote	qwen3:8b	0	0	20871
4	novelty	ollama_remote	qwen3:8b	0	0	17730
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16704
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13432
7	judge	ollama_remote	qwen3:8b	0	0	17253

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 192139 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/567149797f34.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/567149797f34.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/567149797f34.tar.gz (if generated)